

Unit B2: Programming (Core)

Sub-topics: B2.1 to B2.5 (SL Content)

1. Syllabus Scope & Learning Outcomes

The "Do" column defines the practical skills required for the exam (Paper 1 & Paper 2).

Syllabus Point	Command Term	Student Expectation (The "Do")
B2.1.1	Construct / Trace	Write and track code using variables (local/global) and standard data types (integer, float, boolean, string).
B2.1.2	Construct	Write code to slice strings, extract characters, and modify text.
B2.1.3	Describe	Explain how try/except blocks prevent program crashes.
B2.1.4	Construct / Use	Create trace tables to find logic errors; use print debugging.
B2.2.1	Compare	Contrast memory usage and flexibility of Arrays vs. Dynamic Lists.
B2.2.2	Construct	Implement 1D and 2D arrays/lists (accessing indices, iterating).
B2.2.3	Explain	Describe Stack operations (Push, Pop, Peek) using LIFO logic.
B2.2.4	Explain	Describe Queue operations

		(Enqueue, Dequeue) using FIFO logic.
B2.3.1 - B2.3.4	Construct	Write correct sequences, selection (if/else), iteration (for/while), and functions.
B2.4.1	Describe	Calculate Big O notation ($O(1)$, $O(n)$, $O(n^2)$), for specific algorithms.
B2.4.2	Construct / Trace	Write and step through Linear Search and Binary Search algorithms.
B2.4.3	Construct / Trace	Write and step through Bubble Sort and Selection Sort algorithms.
B2.5.1	Construct	Write code to open, read, write, and append to text files.

2. Key Terminology & Definitions

- **Variable Scope:** The region of code where a variable is visible (Local = inside function; Global = everywhere).
- **Identifier:** The user-defined name for a variable, function, or data structure.
- **Truncation:** Removing the fractional part of a number (e.g., in integer division).
- **Concatenation:** Joining two strings together.
- **Static Data Structure:** Fixed size defined at compile time (e.g., standard Array).
- **Dynamic Data Structure:** Can shrink or grow during runtime (e.g., Python List).
- **LIFO (Last In First Out):** The principle governing Stacks (last item added is first removed).
- **FIFO (First In First Out):** The principle governing Queues (first item added is first removed).
- **Algorithm Efficiency:** Measured by Time Complexity (speed) and Space Complexity (memory).

3. Core Content & Exam Actions

B2.1: Fundamentals

Theory:

- **Data Types:** Integer, Float (Double), Boolean, String, Char.
- **Operators:** // (integer division), % (modulus/remainder), != (not equal).
- **String Manipulation:** Slicing strings (e.g., text[0:3]), finding length, concatenating.

Student Exam Action:

- **Tracing:** Given a code snippet with variables updating inside a loop, fill out a table showing the value of every variable at every step.
- **Coding:** Write a script that accepts user input, casts it to the correct type (e.g., int()), and performs a calculation.

B2.2: Data Structures

Theory:

- **1D Arrays/Lists:** Zero-indexed sequences.
- **2D Arrays/Lists:** Grids accessed via [row][column]. Nested loops are required to traverse them.
- **Stacks (LIFO):** Used for "Undo" buttons, browser history, and recursion stacks. Operations: push(), pop(), isEmpty().
- **Queues (FIFO):** Used for printer spooling and keyboard buffers. Operations: enqueue(), dequeue().

Student Exam Action:

- **Comparison:** Compare Static vs. Dynamic structures (Static = fixed memory/fast access; Dynamic = flexible memory/slower access).
- **2D Traversal:** Write a nested for loop to sum all numbers in a 2D grid.

B2.3: Constructs

Theory:

- **Selection:** if, elif, else.
- **Iteration:**
 - **Count-controlled:** for i in range(n) (used when iterations are known).
 - **Pre-condition:** while condition: (used when iterations are unknown).
- **Modularity:** Breaking code into functions (def). Improves maintainability and reusability.

Student Exam Action:

- **Refactoring:** Take a block of repetitive code and rewrite it as a reusable function with parameters.

B2.4: Algorithms (Non-Recursive)

Theory:

- **Big O Notation:**

- $O(1)$: Constant (accessing array index).
- $O(n)$: Linear (Linear Search).
- $O(\log n)$: Logarithmic (Binary Search).
- $O(n^2)$: Quadratic (Bubble/Selection Sort).
- **Linear Search:** Checks every item. Works on unsorted lists.
- **Binary Search:** Checks middle item, discards half. **Requires sorted list.**
- **Bubble Sort:** Swaps adjacent pairs.
- **Selection Sort:** Finds minimum, swaps it to the front.

Student Exam Action:

- **Tracing:** Given a list [5, 2, 9, 1], show the state of the list after *two passes* of Bubble Sort.
- **Coding:** Write a Linear Search function that returns the index of a target.

B2.5: File Processing

Theory:

- **Modes:** Read (r), Write (w - overwrites), Append (a - adds to end).
- **Process:** Open -> Process -> Close (vital to prevent corruption).

Student Exam Action:

- **Coding:** Write a script that reads names.txt line-by-line and prints names longer than 5 characters.

4. Common Pitfalls & Examiner Tips

- **Pitfall (Binary Search):** Students forget that Binary Search **only** works on sorted lists. If the list is unsorted, you must sort it first.
- **Pitfall (2D Arrays):** Confusing [row][column] with [x][y]. In programming, the first index is usually the row (vertical position), not the x-coordinate.
- **Tip (Trace Tables):** If a loop condition is $i < 5$, the loop runs for 0, 1, 2, 3, 4. It does *not* run for 5. Pay close attention to $<$ vs $\leq$.
- **Tip (Pseudocode):** You can use Python syntax in exams, but ensure your indentation is extremely clear.